MCA207P: Web Technology Lab

PROGRAM 1:

Develop and demonstrate the usage of inline and external style sheet using CSS.

1. Inline Stylesheet

```
<!DOCTYPE html>
<html>
<head>
   <title>Inline CSS Example</title>
</head>
<body>
   <h1 style="color: blue; font-family: Arial;">This is an inline styled heading</h1>
   <p style="font-size: 18px; background-color: lightgray;">This paragraph has inline styles applied.</p>
</body>
</html>
```

2. External Stylesheet

styles.css (External CSS file):

```
body {
background-color: #f0f0f0;
font-family: Verdana, sans-serif;
}

h1 {
```

```
color: green;
text-align: center;
}

p {
font-size: 16px;
line-height: 1.5;
}

.highlight {
color: purple;
font-weight: bold;
}
```

index.html (HTML file):

```
<!DOCTYPE html>
<html>
<head>
<title>External CSS Example</title>
<link rel="stylesheet" href="styles.css">
</head>
<body>
<h1>Welcome to External CSS</h1>
<p>This is a paragraph styled by an external stylesheet.</p>
<p class="highlight">This paragraph uses a class defined in the external
stylesheet.</p>
</body>
</html>
```

PROGRAM 2:

Create a table of a minimum size of 4 rows and 4 columns (including headers) using HTML.

Add styles to borders, and cells using CSS. Add a button on the page to hide and show the table.

html

```
<!DOCTYPE html>
<html>
<head>
<title>Table with Hide/Show Button</title>
<link rel="stylesheet" href="style.css">
</head>
<body>

<h1>My Stylish Table</h1>

<button onclick="toggleTable()">Toggle Table</button>

<table id="myTable">
 <thead>
  <tr>
   <th>Header 1</th>
   <th>Header 2</th>
   <th>Header 3</th>
   <th>Header 4</th>
  </tr>
 </thead>
```

```
  <tbody>
    <tr>
      <td>Row 1, Cell 1</td>
      <td>Row 1, Cell 2</td>
      <td>Row 1, Cell 3</td>
      <td>Row 1, Cell 4</td>
    </tr>
    <tr>
      <td>Row 2, Cell 1</td>
      <td>Row 2, Cell 2</td>
      <td>Row 2, Cell 3</td>
      <td>Row 2, Cell 4</td>
    </tr>
    <tr>
      <td>Row 3, Cell 1</td>
      <td>Row 3, Cell 2</td>
      <td>Row 3, Cell 3</td>
      <td>Row 3, Cell 4</td>
    </tr>
  </tbody>
</table>

<script>
function toggleTable() {
  var table = document.getElementById("myTable");
  if (table.style.display === "none") {
    table.style.display = "table"; // Or "block" if you prefer different display
behavior
```

```
  } else {
    table.style.display = "none";
  }
}
</script>

</body>
</html>
```

CSS (style.css)

```
table {
  border-collapse: collapse;  /* Collapses adjacent borders into a single border */
  width: 80%;  /* Adjust table width as desired */
  margin: 20px auto;  /* Center the table and provide some spacing */
}

th, td {
  border: 1px solid #ddd;    /* 1-pixel solid border with a light gray color */
  padding: 8px;      /* Add padding to cells for better readability */
  text-align: left;    /* Align text to the left within cells */
}

th {
  background-color: #f2f2f2;    /* Light gray background for header cells */
  font-weight: bold;     /* Make header text bold */
```

```
}
/* Optional: Zebra-striping for rows */
tr:nth-child(even) {
  background-color: #f9f9f9;    /* Alternate row background color */
}

button {
  display: block;   /* Place the button on a new line */
  margin: 10px auto;   /* Center the button and provide spacing */
  padding: 10px 20px;   /* Add padding to the button */
  background-color: #007bff;   /* Blue background for the button */
  color: white;      /* White text color for the button */
  border: none;      /* Remove default button border */
  cursor: pointer;      /* Change cursor to a pointer on hover */
  border-radius: 5px;   /* Rounded corners for the button */
}

button:hover {
  background-color: #0056b3; /* Darker blue on hover */
}
```

PROGRAM 3: Create an application with different inputs such as radio buttons, date picker, numeric, etc., and display the entered values right below the form.

HTML Structure (index.html):

```
<!DOCTYPE html>
<html>
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>HTML Form</title>
<style>
  body {
    display: flex;
    justify-content: center;
    align-items: center;
    height: 100vh;
    margin: 0;
    background-color: #f0f0f0;
  }
  form {
    width: 400px;
    background-color: #fff;
    padding: 20px;
    border-radius: 8px;
```

```
    box-shadow: 0 0 10px rgba(0, 0, 0, 0.1);
}
fieldset {
    border: 1px solid black;
    padding: 10px;
    margin: 0;
}
legend {
    font-weight: bold;
    margin-bottom: 10px;
}
label {
    display: block;
    margin-bottom: 5px;
}
input[type="text"],
input[type="email"],
input[type="password"],
textarea,
input[type="date"] {
    width: calc(100% - 20px);
    padding: 8px;
```

```
        margin-bottom: 10px;
        box-sizing: border-box;
        border: 1px solid #ccc;
        border-radius: 4px;
    }
    .gender-group {
        margin-bottom: 10px;
    }
    .gender-group label {
        display: inline-block;
        margin-left: 10px;
    }
    input[type="radio"] {
        margin-left: 10px;
        vertical-align: middle;
    }
    input[type="submit"] {
        padding: 10px 20px;
        border-radius: 5px;
        cursor: pointer;
    }
</style>
```

```
</head>
<body>
   <form>
      <fieldset>
         <legend>User Personal Information</legend>
         <label for="name">Enter your full name:</label>
         <input type="text" id="name" name="name" required />
         <label for="email">Enter your email:</label>
         <input type="email" id="email" name="email" required />
         <label for="password">Enter your password:</label>
         <input type="password" id="password" name="pass" required />
         <label for="confirmPassword">Confirm your password:</label>
         <input type="password" id="confirmPassword" name="confirmPass" required
/>
         <label>Enter your gender:</label>
         <div class="gender-group">
            <input type="radio" name="gender" value="male" id="male" required />
            <label for="male">Male</label>
            <input type="radio" name="gender" value="female" id="female" />
            <label for="female">Female</label>
            <input type="radio" name="gender" value="others" id="others" />
            <label for="others">Others</label>
         </div>
```

```
        <label for="dob">Enter your Date of Birth:</label>
        <input type="date" id="dob" name="dob" required />
        <label for="address">Enter your Address:</label>
        <textarea id="address" name="address" required></textarea>
        <input type="submit" value="Submit" />
    </fieldset>
</form>
```

PROGRAM 4 : Develop a HTML Form, which accepts any Mathematical expression. Write JavaScript code to Evaluates the expression and Displays the result.

```
<html>
<head>
<title>Mathematical Expression</title>
<script type = "text/javascript">
   function math_exp()
   {
      var x = document.form1.exptext.value;
      var result = eval(x);
      document.form1.resulttext.value = result;
   }
</script>
</head>

<body >
   <form name = "form1">
      <table>
         <tr>
            <td>Expression</td>
            <td><input type = "text" name = "exptext" /></td>
         </tr>

          <tr>
            <td>Result</td>
```

```
            <td><input type = "text" name = "resulttext" /></td>
        </tr>

        <tr>
            <td><input type = "button" value = "calculate" onclick = "math_exp()"
/></td>
        </tr>

    </table>
  </form>
</body>
</html>
```

PROGRAM 5 : .Create an employee registration form and validate the fields using JavaScript: date, confirm password, password strength, email, number and display custom error messages and success indicators next to each field.

Step 1) Add HTML:

Step 2) Add CSS: Style the input fields and the message box:

Step 3) Add JavaScript:

```
<!DOCTYPE html>
<html>
<head>
<meta name=”viewport” content=”width=device-width, initial-scale=1”>

/* STEP 2 :  Style all input fields */
<style>
Input {
  Width: 100%;
  Padding: 12px;
  Border: 1px solid #ccc;
  Border-radius: 4px;
  Box-sizing: border-box;
  Margin-top: 6px;
  Margin-bottom: 16px;
}
```

```
/* Style the submit button */
Input[type=submit] {
  Background-color: #04AA6D;
  Color: white;
}

/* Style the container for inputs */
.container {
  Background-color: #f1f1f1;
  Padding: 20px;
}

/* The message box is shown when the user clicks on the password field */
#message {
  Display:none;
  Background: #f1f1f1;
  Color: #000;
  Position: relative;
  Padding: 20px;
  Margin-top: 10px;
}



#message p {
  Padding: 10px 35px;
  Font-size: 18px;
```

```
}
/* Add a green text color and a checkmark when the requirements are right */
.valid {
  Color: green;
}
.valid:before {
  Position: relative;
  Left: -35px;
  Content: "✓";
}
/* Add a red text color and an "x" when the requirements are wrong */
.invalid {
  Color: red;
}
.invalid:before {
  Position: relative;
  Left: -35px;
  Content: "✕";
}
</style>
</head>
<body>

<h3>Password Validation</h3>
<p>Try to submit the form.</p>
<div class="container">
  <form action="/action_page.php">
```

```
    <label for=”usrname”>Username</label>
    <input type=”text” id=”usrname” name=”usrname” required>
    <label for=”psw”>Password</label>
    <input type=”password” id=”psw” name=”psw” pattern=”(?=.*\d)(?=.*[a-z])(?=.*[A-Z]).{8,}” title=”Must contain at least one number and one uppercase and lowercase letter, and at least 8 or more characters” required>
    <input type=”submit” value=”Submit”>
  </form>
</div>

<div id=”message”>
  <h3>Password must contain the following:</h3>
  <p id=”letter” class=”invalid”>A <b>lowercase</b> letter</p>
  <p id=”capital” class=”invalid”>A <b>capital (uppercase)</b> letter</p>
  <p id=”number” class=”invalid”>A <b>number</b></p>
  <p id=”length” class=”invalid”>Minimum <b>8 characters</b></p>
</div>

        //STEP 3 : Java Script
<script>
Var myInput = document.getElementById(“psw”);
Var letter = document.getElementById(“letter”);
Var capital = document.getElementById(“capital”);
Var number = document.getElementById(“number”);
Var length = document.getElementById(“length”);

// When the user clicks on the password field, show the message box
```

```
myInput.onfocus = function() {
  document.getElementById(“message”).style.display = “block”;
}
// When the user clicks outside of the password field, hide the message box
myInput.onblur = function() {
 document.getElementById(“message”).style.display = “none”;
}
// When the user starts to type something inside the password field
myInput.onkeyup = function() {

  // Validate lowercase letters
  Var lowerCaseLetters = /[a-z]/g;
  If(myInput.value.match(lowerCaseLetters)) {
   Letter.classList.remove(“invalid”);
    Letter.classList.add(“valid”);
  } else {
 Letter.classList.remove(“valid”);
    Letter.classList.add(“invalid”);
  }

  // Validate capital letters
  Var upperCaseLetters = /[A-Z]/g;
  If(myInput.value.match(upperCaseLetters)) {
    Capital.classList.remove(“invalid”);
    Capital.classList.add(“valid”);
  } else {
    Capital.classList.remove(“valid”);
```

```
    Capital.classList.add(“invalid”);
  }
  // Validate numbers
  Var numbers = /[0-9]/g;
  If(myInput.value.match(numbers)) {
    Number.classList.remove(“invalid”);
    Number.classList.add(“valid”);
  } else {
    Number.classList.remove(“valid”);
    Number.classList.add(“invalid”);
  }
  // Validate length
  If(myInput.value.length >= 8) {
    Length.classList.remove(“invalid”);
Length.classList.add(“valid”);
  } else {
  Length.classList.remove(“valid”);
    Length.classList.add(“invalid”);
  }
}
</script>
</body>
</html>
```

PROGRAM : 6

6. Create a form for Student information. Write JavaScript code to find Total, Average, Result and Grade.

```
<!DOCTYPE html>
<html>
<head>
<title>Registration Form</title>
<script type = “text/javascript”>
   Function calc()
   {
      Var m1,m2,m3,avg = 0,total = 0, result = “”,grade = “”;
      M1 = parseInt(document.form1.wp.value);
      M2 = parseInt(document.form1.sp.value);
      M3 = parseInt(document.form1.cg.value);
      Total = m1+m2+m3;
      Avg = total/3;
      If( m1 < 35 || m2 < 35 || m3 < 35)
      {
        Result = “fail”;
         Grade = “D”;
      }
```

```
Else if(avg >= 75)
{
   Result = “Distinction”;
   Grade = “A+”;
}
Else if(avg >= 60 && avg < 75)
{
  Result = “First class”;
   Grade = “A”;
}
Else if(avg >= 50 && avg < 60)
{
   Result = “Second class”;
   Grade = “B”;
}
Else if(avg >=35 && avg < 50)
{
   Result = “Pass class”;
  Grade = “C”;
}
Else  if (avg < 35)
{
   Result = “Fail”;
   Grade = “D”;
}
Document.form1.result.value = result;
Document.form1.grade.value = grade;
```

```
        Document.form1.total.value = total;
        Document.form1.average.value = avg
    }
</script>
</head>
<body>
    <form name = “form1”>
        <table border = “1”>
            <tr>
                <td> Student Name</td>
                <td><input type = “text”  /></td>
            </tr>
            <tr>
                <td colspan = “2” align = “center”>Subject Marks</td>
            </tr>
            <tr>
                <td>Web Programming</td>
               <td><input type = “text” name = “wp” /></td>
            </tr>
            <tr>
                <td>Computer Graphics</td>
                <td><input type = “text” name = “cg” /></td>
            </tr>
            <tr>
                <td>System Programming</td>
                <td><input type = “text” name = “sp” /></td>
            </tr>
```

```
        <tr>
            <td colspan = “2” align = “center”><input type = “button”   onclick = “calc()” value = “calculte” /></td>
        </tr>
        <tr>
            <td>Total</td>
            <td><input type = “text” name = “total”/></td>
        </tr>
        <tr>
            <td>Average</td>
            <td><input type = “text” name = “average” /></td>
        </tr>
        <tr>
            <td>Result</td>
            <td><input type = “text” name = “result” /></td>
        </tr>
        <tr>
            <td>Grade</td>
            <td><input type = “text” name = “grade”/></td>
        </tr>
      </table>
   </form>
</body>
</html>
```

PROGRAM 7 :

Create a form for Employee information. Write JavaScript code to find DA, HRA, PF, TAX, Gross pay, Deduction and Net pay.

```
<html>
<head>
<title>Registration Form</title>
<script type = “text/javascript”>
   Function calc()
   {
      Var bp,DA,HRA,GP,PF,Tax,Deduction,NetPay,name,id,desg;
      Name = document.form1.firstname.value;
      Id = document.form1.userid.value;
      Desg = document.form1.designation.value;
      Bp = parseInt(document.form1.bp.value);
      DA = bp * 0.5;
      HRA = bp * 0.5;
      GP = bp + DA + HRA;
      PF = GP * 0.02;
      Tax = GP * 0.01;
```

```
        Deduction = Tax + PF;
        NetPay = GP – Deduction;
        Document.form1.da.value = DA;
        Document.form1.hra.value = HRA;
        Document.form1.gp.value = GP;
        Document.form1.pf.value = PF;
        Document.form1.tax.value = Tax;
        Document.form1.deduction.value = Deduction;
        Document.form1.netpay.value = NetPay
    }

</script>
</head>
<body >
    <form name = “form1”>
        <table border = “1”>
            <tr>
                <td>Name</td>
                <td><input type = “text” name = “firstname” /></td>
            </tr>
            <tr>
                <td>User ID</td>
                <td><input type = “text” name = “userid” /></td>
            </tr>
            <tr>
                <td>Designation</td>
                <td><input type = “text” name = “designation” /></td>
```

```
        </tr>
        <tr>
            <td>Basic Pay</td>
            <td><input type = “text” name = “bp”></td>
        </tr>
        <tr>
           <td colspan = “2” align = “center”>
            <input type = “button” name = “calculate” value = “Click Here To Calculate”onclick =”calc()”></td>
        </tr>
        <tr>
            <td>Dearness Allowance </td>
            <td><input type = “text” name = “da”/></td>
        </tr>
        <tr>
            <td>House Rent Allowance </td>
            <td><input type = “text” name = “hra”></td>
        </tr>
        <tr>
            <td>GP</td>
            <td><input type = “text” name = “gp”></td>
        </tr>
        <tr>
            <td>Provident Fund</td>
            <td><input type = “text” name = “pf” /></td>
        </tr>
        <tr>
```

```
            <td>Tax</td>
            <td><input type = “text” name = “tax” /></td>
        </tr>
        <tr>
            <td>Deduction</td>
            <td><input type = “text” name = “deduction” /></td>
        </tr>
        <tr>
            <td>NetPay</td>
            <td><input type = “text” name = “netpay” /></td>
        </tr>
    </table>
  </form>
</body>
</html>
```

PROGRAM 8 :

Develop a to-do list application that allows users to add, remove, and reorder tasks using dragand-drop. Implement event handling for user interactions and validation to prevent duplicate tasks.

1.HTML Structure (index.html):

2.CSS Styling (style.css):

3.JavaScript Functionality (script.js):

1.HTML Structure (index.html):

```
<!DOCTYPE html>
<html lang=”en”>
<head>
<meta charset=”UTF-8”>
<meta name=”viewport” content=”width=device-width, initial-scale=1.0”>
<title>To-Do List</title>
<link rel=”stylesheet” href=”style.css”>
</head>
<body>
<div class=”container”>
```

```
<h1>My To-Do List</h1>
<div class=”input-area”>
<input type=”text” id=”taskInput” placeholder=”Add a new task…”>
<button id=”addTaskBtn”>Add Task</button>
</div>
<ul id=”taskList”>
<!—Tasks will be added here →
</ul>
</div>
<script src=”script.js”></script>
</body>
</html>
```

2.CSS Styling (style.css):

```
Body {
Font-family: sans-serif;
Background-color: #f4f4f4;
Display: flex;
Justify-content: center;
Align-items: center;
Min-height: 100vh;
Margin: 0;
}
.container {
Background-color: #fff;
Padding: 20px;
```

```
Border-radius: 8px;
Box-shadow: 0 2px 10px rgba(0, 0, 0, 0.1);
Width: 400px;
}
.input-area {
Display: flex;
Margin-bottom: 20px;
}

#taskInput {
Flex-grow: 1;
Padding: 10px;
Border: 1px solid #ddd;
Border-radius: 4px;
}
#addTaskBtn {
Padding: 10px 15px;
Background-color: #007bff;
Color: white;
Border: none;
Border-radius: 4px;
Cursor: pointer;
Margin-left: 10px;
}

#taskList {
List-style: none;
```

```
Padding: 0;
}
#taskList li {
Background-color: #f9f9f9;
Padding: 10px;
Margin-bottom: 8px;
Border-radius: 4px;
Display: flex;
Justify-content: space-between;
Align-items: center;
Cursor: grab; /* Indicates draggable */
}
#taskList li.dragging {
Opacity: 0.5;
}
.remove-btn {
Background-color: #dc3545;
Color: white;
Border: none;
Padding: 5px 10px;
Border-radius: 4px;
Cursor: pointer;
}
```

3.JavaScript Functionality (script.js):

```
Document.addEventListener('DOMContentLoaded', () => {
```

```
Const taskInput = document.getElementById(‘taskInput’);
Const addTaskBtn = document.getElementById(‘addTaskBtn’);
Const taskList = document.getElementById(‘taskList’);
Let tasks = JSON.parse(localStorage.getItem(‘tasks’)) || [];
Let draggedItem = null;
Function saveTasks() {
localStorage.setItem(‘tasks’, JSON.stringify(tasks));
}
Function renderTasks() {
taskList.innerHTML = ‘’;
tasks.forEach((taskText, index) => {
const listItem = document.createElement(‘li’);
listItem.textContent = taskText;
listItem.setAttribute(‘draggable’, ‘true’);
listItem.dataset.index = index;
const removeBtn = document.createElement(‘button’);
removeBtn.textContent = ‘Remove’;
removeBtn.classList.add(‘remove-btn’);
removeBtn.addEventListener(‘click’, () => removeTask(index));
listItem.appendChild(removeBtn);
taskList.appendChild(listItem);
});
}
Function addTask() {
Const taskText = taskInput.value.trim();
If (taskText === ‘’) {
Alert(‘Task cannot be empty!’);
```

```
Return;
}
If (tasks.includes(taskText)) {
Alert('This task already exists!');
Return;
}
Tasks.push(taskText);
taskInput.value = '';
saveTasks();
renderTasks();
}
Function removeTask(index) {
Tasks.splice(index, 1);
saveTasks();
renderTasks();
}

// Drag and Drop Event Handlers
taskList.addEventListener('dragstart', € => {
draggedItem = e.target;
setTimeout(() => {
e.target.classList.add('dragging');
}, 0);
});
taskList.addEventListener('dragend', € => {
e.target.classList.remove('dragging');
draggedItem = null;
```

```
});
taskList.addEventListener('dragover', € => {
e.preventDefault();
const afterElement = getDragAfterElement(taskList, e.clientY);
const currentDraggedItem = document.querySelector('.dragging');
if (afterElement == null) {
taskList.appendChild(currentDraggedItem);
} else {
taskList.insertBefore(currentDraggedItem, afterElement);
}
});
taskList.addEventListener('drop', () => {
if (!draggedItem) return;
const oldIndex = parseInt(draggedItem.dataset.index);
const newIndex = Array.from(taskList.children).indexOf(draggedItem);

// Update the tasks array based on the new order
Const [movedTask] = tasks.splice(oldIndex, 1);
Tasks.splice(newIndex, 0, movedTask);
saveTasks();
renderTasks(); // Re-render to update data-index attributes
});
Function getDragAfterElement(container, y) {
Const draggableElements = [...container.querySelectorAll('li:not(.dragging)')];
Return draggableElements.reduce((closest, child) => {
Const box = child.getBoundingClientRect();
Const offset = y – box.top – box.height / 2;
```

```
If (offset < 0 && offset > closest.offset) {
Return { offset: offset, element: child };
} else {
Return closest;
}
}, { offset: Number.NEGATIVE_INFINITY }).element;
}
addTaskBtn.addEventListener(‘click’, addTask);
taskInput.addEventListener(‘keypress’, € => {
if (e.key === ‘Enter’) {
addTask();
}
});
renderTasks(); // Initial rendering of tasks
});
```

PROGRAM 9:

Create an AngularJS app where users can input two numbers and select an operation to perform. The result should be displayed dynamically.

Index.html:

```
<!DOCTYPE html>
<html ng-app=”calculatorApp”>
<head>
        <title>AngularJS Calculator</title>
<script src=https://ajax.googleapis.com/ajax/libs/angularjs/1.8.2/angular.min.js></script>
<script src=”app.js”></script>
</head>
<body>
<div ng-controller=”CalculatorController”>
<h1>Simple Calculator</h1>
```

```
<input type=”number” ng-model=”num1” placeholder=”Enter first number”>
<input type=”number” ng-model=”num2” placeholder=”Enter second number”>
<select ng-model=”operation”>
<option value=”+”>Add (+)</option>
<option value=”-“>Subtract (-)</option>
<option value=”*”>Multiply (*)</option>
<option value=”/”>Divide (/)</option>
</select>
<p>Result: {{ calculateResult() }}</p>
</div>
</body>
</html>
```

App.js:

JavaScript

```
Angular.module(‘calculatorApp’, [])
  .controller(‘CalculatorController’, function($scope) {
      $scope.num1 = 0;
      $scope.num2 = 0;
      $scope.operation = ‘+’;
      $scope.calculateResult = function() {
          Var result;
          Var n1 = parseFloat($scope.num1);
          Var n2 = parseFloat($scope.num2);
```

```
        If (isNaN(n1) || isNaN(n2)) {
            Return “Invalid input”;
        }
        Switch ($scope.operation) {
            Case ‘+’:
                Result = n1 + n2;
                Break;
            Case ‘-‘:
                Result = n1 – n2;
                Break;
            Case ‘*’:
                Result = n1 * n2;
                Break;
            Case ‘/’:
                If (n2 === 0) {
                    Return “Cannot divide by zero”;
                }
                Result = n1 / n2;
                Break;
            Default:
                Result = “Invalid operation”;
        }
        Return result;
    };
});
```

PROGRAM 11:

Write an AJAX implementation where the content of a text file will be displayed on some part of the page without altering the existing text or refreshing the page. Instead, use a button to update the contents.

Method:

To implement an AJAX solution that displays the content of a text file on a part of a page without altering existing text or refreshing the page, and updates it with a button, follow these steps:

1. Create the HTML Structure:

Create an HTML file with a designated area to display the text file's content and a button to trigger the update.

2. Create the Text File:Create a simple text file (e.g., mytextfile.txt) in the same directory as your HTML file.
3. Implement the JavaScript (AJAX):

Create a script.js file and add the following JavaScript code. This code will handle the AJAX request to fetch the content of mytextfile.txt and update the contentDisplay div.

1. Create the HTML Structure:

Create an HTML file with a designated area to display the text file's content and a button to trigger the update.

```
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1.0">
<title>AJAX Text File Display</title>
</head>
<body>
<h1>My Page Content</h1>
<p>This is some existing text on the page that should not be altered.</p>
<div id="contentDisplay">
<!—Text file content will be loaded here →
Loading content…
</div>
<button id="updateContentBtn">Update Content</button>
<script src="script.js"></script>
</body>
</html>
```

2. Create the Text File:

Create a simple text file (e.g., mytextfile.txt) in the same directory as your HTML file.

```
This is the content from mytextfile.txt.
```

It can be updated dynamically.

3. Implement the JavaScript (AJAX):

Create a script.js file and add the following JavaScript code. This code will handle the AJAX request to fetch the content of mytextfile.txt and update the contentDisplay div.

JavaScript

```
Document.addEventListener('DOMContentLoaded', function() {
  Const updateButton = document.getElementById('updateContentBtn');
  Const contentDisplay = document.getElementById('contentDisplay');
  updateButton.addEventListener('click', function() {
    const xhr = new XMLHttpRequest();
    xhr.open('GET', 'mytextfile.txt', true); // true for asynchronous
    xhr.onload = function() {
      if (xhr.status === 200) {
        // Request was successful
        contentDisplay.textContent = xhr.responseText;
      } else {
        // There was an error
        contentDisplay.textContent = 'Error loading content: ' + xhr.status;
      }
    };
   Xhr.onerror = function() {
```

```
        // Network error
        contentDisplay.textContent = 'Network error during content loading.';
    };
    Xhr.send();
  });
});
```